Bounded Tool-Calling Agent

Agentic AI | Implementation and interview guide

Purpose and intended demonstration

Execute a model-proposed tool plan through an allowlisted dispatcher with strict budgets and an audit trail.

How the implementation works

Live mode requests a JSON plan from a local LLM; fixture mode replays a supplied plan. Validate a maximum of four calls, permitted tools and exact argument schemas. A calculator parses a restricted arithmetic AST; document search uses the local corpus.

Evaluation and evidence

Test unknown tools, excessive call counts and unsafe calculator expressions. Execute the sample plan and record each tool observation. This is plan-then-execute orchestration, not an autonomous open-ended agent.

Bounded Tool-Calling Agent

Agentic AI | Implementation and interview guide

Setup and commands

Requires Python 3.10 or newer and NumPy. Create a virtual environment: python -m venv .venv. On Windows use .venv\Scripts\python.exe if activation is unavailable. Install with python -m pip install -r requirements.txt. Run python app.py, then open results/report.html in your browser. Run python -m unittest discover -v to execute tests. The default seed is 42; change it with --seed. Replace sample_input.json using --input path-to-file.json, preserving its schema. Algorithm-only projects generate synthetic arrays from the seed rather than consuming external files. For actual language-model generation, install and start Ollama, install a model appropriate for your hardware, then run python app.py --mode live --model YOUR_INSTALLED_MODEL. The local endpoint is http://127.0.0.1:11434/api/generate. A model download and additional memory are required; none were installed or tested for this build. Live requests time out after 120 seconds and failures are not silently replaced by fixture output.

Architecture and file responsibilities

CLI arguments -> JSON input or seeded generator -> domain pipeline -> validation or evaluation -> JSON and HTML artifacts. Imported functions can be used in a notebook or a separate application. Model checkpoints preserve learned arrays where applicable; see app.py for parameter order. This package does not include a production web service. Each project is independent and includes its own helper code so the ZIP can be extracted and run alone. app.py holds the experiment and domain functions: calculate, execute, run. kit.py contains reporting, input loading and only the retrieval or generation helpers needed by this project. sample_input.json contains editable inputs. test_app.py checks domain behaviors; test_reporting.py checks metrics and HTML output boundaries. results/metrics.json records outputs and results/report.html provides a browser-readable report. Numeric projects also save arrays or learned parameters to results/artifacts.npz; inspect its keys with numpy.load. MODEL_CARD.md explains intended use and limits. .github/workflows/tests.yml runs tests and the fixture experiment. No API key is needed.

Limitations and next improvements

The planner is not live-tested in this build. Tool permission checks are real, but no browser, shell, network search or filesystem-writing tool is provided. Add observation-conditioned replanning, per-tool deadlines, human confirmation for side effects, budget accounting and adversarial plan evaluations.

Interview walkthrough

Present the problem and demonstrate one normal example, then one failure or boundary case. Explain the baseline or fixture mode, the split or validation design, and one limitation before discussing an extension. Be ready to trace data through the functions listed above. Explain why the chosen metric matches the task and why the available results cannot establish production performance. Add your own documented experiment and be precise about what you implemented or changed. These are AI-assisted educational projects; filenames and synthetic scores are not evidence of client work.

Verified build results

This build verified fixture replay and validation only. No LLM weights were downloaded and live generation quality was not measured.

project

"Bounded Tool-Calling Agent"

mode

"fixture"

task

"Find the refund policy and calculate 125 times 3 plus 20."

tools used

2

budget

4

See results/metrics.json for all observations and TEST_RESULTS.txt for the executed tests. All datasets and replay responses shipped here are synthetic. Results can vary across seeds and numerical library versions.

Provider references: https://docs.ollama.com/api/generate and https://docs.ollama.com/capabilities/structured-outputs. The adapter uses non-streaming generation and requests JSON schema output where applicable. Independent application validation still runs after generation.